

STRIPE PROJECT

3. NoSQL Data System

D4. NoSQL Design Diagrams

Nicolas Pichon - AIA RNCP 38777 / BC02 / D4 : NoSQL Design Diagrams / v25 - 2026/10/13.

3.0. Contexte

Le *business case* Stripe demande de couvrir les données non structurées et semi-structurées issues des sources suivantes :

- interactions utilisateur (clickstreams et données de session),
- features de machine learning,
- retours clients,
- journaux.

Compte tenu des exigences du *business case* (TR1, en particulier), le choix d'un système de stockage *par document* s'impose (cf. justification en annexe 3.A).

3.1. Principes de conception

Le modèle *NoSQL* est gouverné par les trois principes suivant.

Modéliser selon les accès, pas selon les entités

En relationnel, on modélise le domaine puis on écrit les requêtes. En document, on part des requêtes : chaque collection est conçue pour qu'une lecture typique n'implique **aucune jointure**.

Schéma flexible mais contrôlé

L'absence de schéma imposé permet d'absorber des structures hétérogènes (un log d'erreur n'a pas les mêmes champs qu'un log d'accès). On applique néanmoins une validation *JSON Schema* sur les champs critiques, pour éviter la dérive silencieuse.

Imbriquer par défaut, référencer par exception

Cf. §3.3. ci-dessous.

3.2. Collections

3.2.1 event_logs - journaux applicatifs

Volume très élevé, écriture massive, lecture par fenêtre temporelle.

```
{
  "_id": ObjectId("..."),
  "timestamp": ISODate("2026-07-23T14:32:11.482Z"),
  "level": "error",
  "service": "payment-gateway",
  "merchant_id": "a3f1...",
  "transaction_id": "b7c2...",
  "message": "Card authorization timeout",
  "context": {
    "http_status": 504,
    "latency_ms": 30012,
    "retry_count": 2,
    "upstream": "acquirer-eu-1"
  },
  "trace_id": "9f8e7d...",
}
```

```
"expires_at": ISODate("2026-10-23T14:32:11.482Z")
}
```

Le sous-document `context` est **volontairement libre** : sa forme varie selon le service émetteur. C'est exactement le cas d'usage où un schéma relationnel forcerait soit une table par type de log, soit une colonne fourre-tout.

- **Clé de partitionnement (shard key)** : { service: 1, timestamp: 1 }
- **Index** : { timestamp: -1 }, { merchant_id: 1, timestamp: -1 }, { transaction_id: 1 } (sparse), { level: 1, timestamp: -1 }
- **TTL** : index sur `expires_at` - purge automatique après 90 jours.

3.2.2 user_sessions - clickstream et interactions

Une session = un document. Les événements sont **imbriqués**, car on ne consulte jamais un clic isolément : on lit toujours la session entière.

```
{
  "_id": "sess_8f2a...",
  "customer_id": "c91d...",
  "merchant_id": "a3f1...",
  "started_at": ISODate("2026-07-23T10:02:00Z"),
  "ended_at": ISODate("2026-07-23T10:19:44Z"),
  "duration_sec": 1064,
  "device": {
    "type": "mobile",
    "os": "iOS 18.2",
    "browser": "Safari",
    "screen": "390x844"
  },
  "geo": {
    "ip_country": "FR",
    "region": "Bretagne",
    "city": "Rennes"
  },
  "events": [
    { "ts": ISODate("2026-07-23T10:02:04Z"), "type": "page_view", "path": "/checkout" },
    { "ts": ISODate("2026-07-23T10:03:12Z"), "type": "click", "target": "add_card" },
    { "ts": ISODate("2026-07-23T10:04:55Z"), "type": "form_error", "field": "cvc" },
    { "ts": ISODate("2026-07-23T10:06:30Z"), "type": "purchase", "transaction_id": "b7c2..." }
  ],
  "outcome": "converted"
}
```

Limite d'imbrication : un document MongoDB plafonne à 16 Mo. Une session très longue pourrait dépasser cette taille - on borne donc `events` (par exemple 1 000 entrées) et on bascule le surplus dans un document `user_sessions_overflow` référencé. C'est le **pattern outlier**.

- **Shard key** : { merchant_id: 1, started_at: 1 } - cloisonnement marchand préservé, comme en OLTP.
- **Index** : { customer_id: 1, started_at: -1 }, { "events.transaction_id": 1 } (multikey), { outcome: 1 }

3.2.3 ml_features - magasin de features

Alimente la détection de fraude en temps réel. Lecture à très faible latence sur clé primaire.

```
{
  "_id": "cust:c91d...",
  "entity_type": "customer",
  "entity_id": "c91d...",
  "merchant_id": "a3f1...",
  "computed_at": ISODate("2026-07-23T14:00:00Z"),
  "features": {
    "txn_count_24h": 7,
    "txn_amount_sum_24h": 842.50,
    "distinct_countries_7d": 3,
    "distinct_cards_30d": 2,
    "avg_ticket_90d": 61.30,
    "failed_ratio_7d": 0.21,
    "hours_since_last_txn": 0.4,
    "device_switch_count_24h": 2
  },
  "model_scores": {
    "fraud_v3": 0.87,
    "churn_v1": 0.12
  },
  "version": "v3.2.1"
}
```

Ce document est le **point d'entrée du scoring temps réel** : une seule lecture par clé suffit pour disposer de toutes les features. Aucune agrégation à la volée, aucune jointure - c'est ce qui rend la décision possible en quelques millisecondes.

- **Shard key** : { _id: "hashed" } - répartition uniforme.
- **Index** : { merchant_id: 1, computed_at: -1 },
{ "model_scores.fraud_v3": -1 }

3.2.4 customer_feedback - avis et enquêtes

Volume modeste, structure hétérogène, recherche plein texte.

```
{
  "_id": ObjectId("..."),
  "merchant_id": "a3f1...",
  "customer_id": "c91d...",
  "type": "survey_response",
  "submitted_at": ISODate("2026-07-20T09:14:00Z"),
  "channel": "email",
  "rating": 4,
  "text": "Le paiement a fonctionné, mais la confirmation a mis du temps.",
  "language": "fr",
  "nlp": {
    "sentiment": "mixed",
    "sentiment_score": 0.12,
    "topics": ["latency", "confirmation"],
    "model_version": "sent-v2"
  },
  "responses": [
    { "question_id": "q1", "answer": "4" },
    { "question_id": "q2", "answer": "Confirmation lente" }
  ]
}
```

- **Index** : { merchant_id: 1, submitted_at: -1 },
index texte sur text, { "nlp.sentiment": 1 }

3.2.5 merchant_config - configuration dénormalisée

Petite collection très lue, quasi jamais écrite. Sert de cache de référence pour éviter d'interroger l'OLTP à chaque événement.

```
{
  "_id": "a3f1...",
  "legal_name": "Librairie du Désert",
  "country_code": "FR",
  "status": "active",
  "risk_profile": {
    "tier": "standard",
    "manual_review_threshold": 0.75,
    "auto_block_threshold": 0.95
  },
  "enabled_features": ["3ds", "subscriptions", "instant_payout"],
  "synced_at": ISODate("2026-07-23T06:00:00Z")
}
```

3.3. Stratégies de relations

Le *business case* demande explicitement de traiter l'imbrication, le référencement et l'indexation. On applique les règles suivantes.

Imbrication (embedding)

Retenue quand les données sont lues ensemble, bornées en taille et appartiennent au parent.

Exemples : les `events` d'une session, les `responses` d'une enquête, le sous-document `device`.

Avantage décisif : une seule lecture disque, aucune jointure. C'est le principal levier de performance en *NoSQL*.

Référencement

Retenu quand l'entité est partagée, volumineuse ou de cycle de vie indépendant.

Exemples : `merchant_id` et `customer_id` référencent les entités maîtres de l'OLTP ; `transaction_id` fait le lien avec le transactionnel.

Point important : ces références sont logiques, pas contraintes. Il n'y a pas d'intégrité référentielle garantie par le moteur. C'est le prix de la flexibilité, et cela impose que l'OLTP reste la source de vérité pour ces entités.

Pattern « extended reference »

Compromis employé dans `event_logs` et `user_sessions` : on stocke la référence (`merchant_id`) plus une copie des quelques attributs fréquemment affichés (le nom du marchand, par exemple). On évite ainsi une lecture supplémentaire, au prix d'une redondance acceptable sur des données qui changent rarement.

Indexation

Trois familles d'index sont utilisées :

Type	Usage dans le modèle
Simple / composé	{ merchant_id: 1, timestamp: -1 } - filtre + tri fréquents

Type	Usage dans le modèle
Multikey	{ "events.transaction_id": 1 } - sur un tableau imbriqué
Texte	customer_feedback.text - recherche plein texte
TTL	event_logs.expires_at - purge automatique
Sparse	{ transaction_id: 1 } - le champ n'existe que sur certains logs

Même arbitrage qu'en *OLTP* : chaque index accélère la lecture et pénalise l'écriture. Sur `event_logs`, qui reçoit un flux massif, on reste volontairement parcimonieux.

3.4. Intégration avec OLTP & OLAP

Le *business case* exige que le modèle *NoSQL* s'intègre aux deux autres systèmes.

OLTP > NoSQL

Les captures *CDC* sur `transaction`, `merchant` et `customer` alimentent un flux *Kafka*. Un consommateur met à jour `merchant_config` et déclenche le recalcul des `ml_features`. Les identifiants *OLTP* (`transaction_id`, `merchant_id`) servent de clés de rapprochement.

NoSQL > OLAP

Les sessions et les scores de fraude sont agrégés par lots nocturnes et chargés dans `fact_fraud_event` et les dimensions comportementales. Les données brutes restent dans le système *NoSQL*, seul l'agrégat part vers l'entrepôt.

NoSQL comme couche de rapprochement transverse

C'est à ce niveau que l'on peut rapprocher une même personne physique agissant chez plusieurs marchands (via l'empreinte de carte ou l'appareil), sans compromettre le cloisonnement transactionnel entre marchands.

3.5. Cohérence, sécurité et conformité

Modèle de cohérence

Cohérence à terme (*eventual consistency*) acceptée pour les logs et les sessions : un délai de quelques secondes est sans conséquence. En revanche, `ml_features` utilise une lecture `readConcern: "majority"` sur le chemin de décision de fraude, où une donnée périmée aurait un coût réel.

Chiffrement

Chiffrement au repos (au niveau du stockage) et en transit (TLS). Les champs sensibles (`text` des retours clients, données d'identification) relèvent du chiffrement au niveau du champ (*Client-Side Field Level Encryption*).

RGPD

Le droit à l'effacement impose de pouvoir supprimer toutes les traces d'un client. Les index sur `customer_id` présents dans chaque collection rendent cette opération réalisable. Les index TTL assurent par ailleurs la limitation de conservation des journaux.

PCI-DSS

Aucune donnée de carte complète n'est stockée dans le système *NoSQL*, conformément au principe déjà appliqué en *OLTP*.

Contrôle d'accès

Rôles par collection (lecture seule sur `event_logs` pour les analystes, écriture réservée aux services), et journalisation des accès.

Annexes

3.A. Justification du modèle de stockage

Pourquoi choisir une base orientée document?

Le cahier des charges (*Stripe business case*) est en réalité assez directif sur ce choix : l'exigence TR1 demande explicitement une base capable de "*efficient querying of nested and unstructured data*". C'est cette exigence précise qui élimine les trois autres familles de stockage NoSQL.

Sources de données

- **Clickstreams des utilisateurs** : un événement de clic n'a pas les mêmes attributs qu'une soumission de formulaire ou qu'une vue de page ; la structure varie par nature d'un enregistrement à l'autre.
- **Données de session des utilisateurs** : idem.
- **Machine Learning features (détection de fraude)** : un vecteur de caractéristiques évolue au fil des itérations du modèle : on ajoute, retire, recombine des variables sans prévenir.
- **Retours clients (avis, enquêtes)** : texte libre plus métadonnées dont les champs varient selon le canal de collecte.

Le point commun de ces sources : une structure imprévisible et changeante, et de gros volumes.

Pourquoi rejeter les trois autres familles de stockage?

Stockage par clé-valeur (*Redis, DynamoDB*)

Excellent pour une lecture par clé unique, mais aucune capacité native à interroger à l'intérieur de la donnée. Impossible de requêter "*tous les événements t.q. device_type = mobile et event_type = click*" sans tout rapatrier du côté de l'application. Or c'est exactement ce type de requête qu'exige l'exigence TR1.

Stockage par colonne (*Cassandra, HBase*)

Très bon pour un débit d'écriture extrême sur un schéma de colonnes relativement stable, mais une famille de colonnes reste rigide par nature. Mal adapté à des enregistrements dont la forme change d'un type d'événement à l'autre.

Stockage en graphe (*Neo4j*)

Pertinent pour des requêtes de traversée de relations (détection de réseaux de fraude coordonnée, par exemple), mais ce n'est pas le besoin ici : on stocke des événements et des vecteurs de caractéristiques, pas un graphe de relations à parcourir.

Ce que le stockage par document apporte spécifiquement

Le format correspond déjà à la donnée source

Un événement de clickstream envoyé par une app web/mobile est nativement un objet JSON. Le stocker tel quel, sans le forcer dans un schéma relationnel fixe, évite une transformation qui n'apporterait rien.

Schéma flexible sans migration

Un nouveau type d'événement, un nouveau champ de feature ML : un document orienté document l'absorbe immédiatement, sans ALTER TABLE ni migration des enregistrements existants. Cohérent avec un système qui, par nature, évolue en continu (nouvelles features testées, nouvelle instrumentation produit).

Requêtage riche sur des champs imbriqués

Indexation secondaire sur des sous-champs, agrégations sur des tableaux imbriqués. C'est exactement ce que demande l'exigence TR1 (et ce qu'un simple stockage clé-valeur ne permet pas).

Passerelle naturelle vers le pipeline déjà conçu

Le choix d'une base orientée documents s'imbrique bien dans l'architecture globale posée en D1 : le mécanisme des *Change Streams* propre aux systèmes de stockage par document (équivalent au mécanisme *CDC* dans un système transactionnel) permettent d'alimenter le pipeline *Airflow* → *OLAP* par le même principe de réplication asynchrone qui alimente *ChangeAuditLog*.

Sharding horizontal natif

Répond à TR3 ("*distributed databases and data sharding*") — un partitionnement par `customer_id` ou `session_id` scale horizontalement sans repenser le modèle.

Glossaire

- Sharding
- Change Streams